Contents

Graph Traversal — BFS & DFS — Complete Interview Handbook

Pattern #18 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — Graph section

---

Table of Contents

1. Pattern Overview · 2. Recognition Guide · 3. Decision Framework · 4. Why This Pattern Works · 5. Algorithm Framework · 6. Time & Space Complexity · 7. Edge Cases · 8. Pros & Cons · 9. Real World Applications · 10. Interview Strategy · 11. Pattern Variations · 12. Comparison With Other Patterns · 13. Problem Classification · 14. 30 Interview Problems · 15. Common Mistakes · 16. Optimization Techniques · 17. Multiple Language Templates · 18. Dry Runs · 19. Advanced Concepts · 20. Senior Engineer Insights · 21. Cheat Sheet · 22. Revision Notes · 23. Practice Roadmap · 24. Memory Tricks · 25. Final Summary

---

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

Graph BFS and DFS generalize tree traversal to structures that may contain **cycles and multiple paths between nodes**. Since a graph isn't guaranteed acyclic, both algorithms require an explicit **visited set** to avoid infinite loops — the core addition that distinguishes graph traversal from tree traversal. BFS explores level-by-level (shortest path in unweighted graphs); DFS explores as deep as possible before backtracking (connectivity, cycle detection, topological structure).

1.2 Why Was This Pattern Invented?

Many real-world structures (social networks, road networks, dependency graphs, web links) aren't trees — they have cycles and multiple paths. Traversal algorithms need a way to systematically visit every reachable node **exactly once**, regardless of how many cycles or alternate paths exist, to answer connectivity, shortest-path, and structural questions correctly and efficiently.

1.3 Real Intuition Behind The Pattern

Imagine **exploring an unfamiliar subway system** — you need to mark stations you've already visited (visited set) so you don't loop endlessly around a circular line; BFS is like checking every station exactly one stop away before checking any two stops away (shortest-hop discovery); DFS is like committing to one line and riding it to its end before backtracking to try a different line.

1.4 Mental Model

"What haven't I visited yet, and how do I reach it?" — every graph traversal maintains a **visited set** to guarantee termination and correctness (each node processed exactly once), plus a frontier structure (queue for BFS, stack/recursion for DFS) determining the *order* of exploration.

1.5 Visual Explanation

```
Graph (undirected):
1 - 2 - 3
|       |
4 - - - 5

BFS from 1:
visited={1}, queue=[1]
dequeue 1 → visit neighbors 2,4 → visited={1,2,4}, queue=[2,4]
dequeue 2 → visit neighbor 3 (4 already visited via 1, skip) → visited={1,2,4,3}, queue=[4,3]
dequeue 4 → visit neighbor 5 (1 already visited) → visited={1,2,4,3,5}, queue=[3,5]
dequeue 3 → neighbor 5 already visited → queue=[5]
dequeue 5 → no new neighbors → queue=[]
BFS order: 1, 2, 4, 3, 5

DFS from 1:
visit 1 → visit 2 → visit 3 → visit 5 (via 3) → visit 4 (via 5) → backtrack, done
DFS order: 1, 2, 3, 5, 4
```

1.6 Simple Analogy

Graph BFS/DFS is like **exploring a maze with cycles by chalk-marking every junction you've already visited** — without the chalk marks (visited set), you'd walk in circles forever; with them, you're guaranteed to explore every reachable room exactly once.

1.7 When Should I Immediately Think About Using This Pattern?

- "Is there a path between A and B?" (connectivity).
- "Shortest path in an **unweighted** graph" (BFS specifically).
- "Number of connected components/islands."
- "Detect a cycle" in a graph.
- Grid problems framed as implicit graphs (each cell is a node, adjacent cells are edges) — "number of islands," "rotting oranges," "flood fill."

SECTION 2 — Recognition Guide

2.1 Keywords

Keyword	Signal
"connected components," "islands"	BFS/DFS for connectivity counting
"shortest path" (unweighted)	BFS specifically
"path exists between," "can reach"	BFS or DFS for connectivity
"flood fill," "rotting oranges"	Grid-as-graph BFS
"detect a cycle"	DFS with color-marking (directed) or Union-Find (undirected)
"clone graph," "all paths from A to B"	DFS with path/state tracking

2.2 Hidden Hints

A 2D grid where movement is restricted to adjacent cells (up/down/left/right) is **always** an implicit graph problem — even if the word "graph" never appears in the problem statement.

2.3 Interview Clues

Interviewer draws a grid or a network diagram with cycles (not a tree) — the presence of cycles is the definitive signal that a visited set is mandatory, unlike tree traversal.

2.4 Common Trick Words

"Number of provinces," "number of islands," "minimum steps to reach," "surrounded regions" — all map directly to connectivity or shortest-path graph traversal.

2.5 What Interviewers Expect

Correct and consistent visited-set usage (marking visited **at enqueue time** for BFS to avoid duplicate enqueuing, not just at dequeue time), correct choice of BFS (shortest path, unweighted) vs DFS (connectivity, cycle detection, exhaustive path enumeration), and correct handling of both directed and undirected graph representations.

2.6 When NOT To Use This Pattern

- The graph has **weighted edges** and you need the shortest path — plain BFS assumes unit edge weights; use Dijkstra's/Bellman-Ford (Pattern #21) instead.
- You need a topological ordering respecting dependencies — while DFS-based topological sort exists, it's a distinct enough concept to warrant its own pattern (Pattern #20).
- You need to detect cycles specifically in an **undirected** graph incrementally as edges are added — Union-Find (Pattern #19) is often more efficient for this incremental scenario than repeated DFS.

SECTION 3 — Decision Framework

```
Is the data a graph (or grid-as-graph) with POSSIBLE CYCLES?

       Yes → Requires an explicit VISITED SET (unlike tree traversal)


Do you need the SHORTEST PATH in an UNWEIGHTED graph, or LEVEL-BY-LEVEL exploration?

       Yes → USE BFS

        No

Do you need CONNECTIVITY, CYCLE DETECTION, or EXHAUSTIVE PATH enumeration?

       Yes → USE DFS

        No

Are edges WEIGHTED and you need shortest path?

       Yes → USE DIJKSTRA'S/BELLMAN-FORD (Pattern #21) instead - BFS assumes unit weights

        No

Do you need a DEPENDENCY-RESPECTING ORDERING (DAG)?

       Yes → USE TOPOLOGICAL SORT (Pattern #20) instead
```

Why: BFS's level-synchronized exploration naturally gives shortest paths **only** when every edge costs the same (unweighted); DFS's depth-first commitment is naturally suited to exhaustive exploration and structural questions (cycles, connectivity) but doesn't guarantee shortest paths. Choosing based on the actual question being asked (distance vs. structure) is the key decision.

SECTION 4 — Why This Pattern Works

Mathematical/Logical: Both BFS and DFS visit each node and edge **at most once** (guaranteed by the visited set), giving O(V + E) time complexity — visiting every vertex once (O(V)) and examining every edge once when checking neighbors (O(E)). The visited set is what prevents the exponential blowup that would otherwise occur from repeatedly re-exploring cycles.

BFS shortest-path correctness proof: *Invariant:* when a node is dequeued, the number of edges traversed to reach it (its BFS "level") equals its true shortest distance from the source in an unweighted graph. *Base case:* the source is at distance 0, trivially true. *Inductive step:* since BFS processes all nodes at distance `d` before any node at distance `d+1` (proven identically to Tree BFS's level-order guarantee, §4 of Pattern #14, but now with a visited set preventing revisiting nodes via longer paths), any node first discovered while processing a distance-`d` node must be at distance exactly `d+1` — it cannot be reached by any shorter path, since all shorter-distance nodes were already fully processed. *Termination:* every reachable node is eventually dequeued with its correct shortest distance. **QED.**

DFS connectivity correctness proof: *Invariant:* a single DFS call from a source visits exactly the set of nodes reachable from that source. *Base case:* the source itself is trivially reachable and

visited first. *Inductive step:* every unvisited neighbor of a visited node is, by definition, reachable, and gets visited by the recursive/iterative DFS call; every already-visited node is correctly skipped (preventing infinite loops on cycles). *Termination:* DFS terminates when no unvisited reachable node remains, having visited exactly the connected component containing the source. **QED.**

SECTION 5 — Algorithm Framework

5.1 Step-by-Step Framework (BFS)

1. Initialize a queue with the source node; mark it visited **immediately upon enqueueing** (not upon dequeueing — a critical subtlety to avoid duplicate enqueues).
2. While the queue is non-empty: dequeue a node, process it.
3. For each unvisited neighbor: mark it visited and enqueue it.
4. Repeat until the queue is empty.

5.2 General Template — BFS

```
function bfs(graph, source):
    visited = {source}
    queue = [source]
    distances = {source: 0}

    while queue is not empty:
        node = queue.dequeue()
        for neighbor in graph.neighbors(node):
            if neighbor not in visited:
                visited.add(neighbor)
                distances[neighbor] = distances[node] + 1
                queue.enqueue(neighbor)

    return distances
```

5.3 General Template — DFS (Recursive)

```
function dfs(graph, node, visited):
    if node in visited: return
    visited.add(node)
    process(node)
    for neighbor in graph.neighbors(node):
        dfs(graph, neighbor, visited)
```

5.4 Cycle Detection Template (Directed Graph, DFS with 3-Color Marking)

```
function hasCycle(graph):
    WHITE, GRAY, BLACK = 0, 1, 2
    color = {node: WHITE for node in graph.nodes}

    function dfs(node):
        color[node] = GRAY                      # currently in recursion stack
        for neighbor in graph.neighbors(node):
            if color[neighbor] == GRAY: return true      # back edge → cycle found
            if color[neighbor] == WHITE and dfs(neighbor): return true
        color[node] = BLACK                      # fully processed
        return false
```

```
for node in graph.nodes:
    if color[node] == WHITE:
        if dfs(node): return true
return false
```

5.5 Interview Thinking Process

1. "This is a graph (or grid-as-graph) that may have cycles — I need an explicit visited set, unlike tree traversal."
2. "If I need shortest path in an unweighted graph, I'll use BFS, since its level-synchronized exploration guarantees the first time I reach a node is via the shortest path."
3. "If I need connectivity or cycle detection, I'll use DFS, marking nodes visited to avoid infinite loops."
4. "For directed-graph cycle detection specifically, I'll use 3-color marking (white/gray/black) to distinguish 'currently being processed' (gray, indicating a back-edge cycle) from 'fully processed' (black)."
5. "I'll state the complexity as O(V + E) for both, since every vertex and edge is examined at most once."

SECTION 6 — Time & Space Complexity

Case	Time	Space	Why
Worst Case	O(V + E)	O(V) for visited set + O(V) queue/stack (BFS) or O(V) recursion depth (DFS)	Every vertex visited once, every edge examined once
Average Case	O(V + E)	O(V)	Same regardless of graph structure
Best Case	O(V + E) (must potentially visit all reachable nodes)	O(V)	Even connectivity checks may need to examine the whole reachable component
Amortized	O(V + E) total across the single traversal	O(V)	No repeated work — visited set ensures each node/edge processed once

Adjacency list vs matrix: using an adjacency list gives O(V + E) traversal; using an adjacency matrix gives $O(V^2)$ traversal (since checking all possible neighbors for each node costs O(V) regardless of actual edge count) — always prefer adjacency lists for sparse graphs.

SECTION 7 — Edge Cases

Edge Case	Example	Handling
Empty graph	No nodes	Return immediately, no traversal needed
Disconnected graph	Multiple components	Must iterate over ALL nodes and start a new BFS/DFS from each unvisited node to cover every component
Self-loops	Node points to itself	Visited-set check correctly prevents infinite re-processing
Multi-edges (parallel edges)	Multiple edges between the same pair of nodes	Doesn't affect correctness, but may cause redundant neighbor-checking — deduplicate the adjacency list if efficiency matters
Directed vs undirected confusion	Treating a directed graph as undirected or vice versa	Always clarify edge directionality explicitly; cycle detection algorithms differ significantly between the two
Single node, no edges	Isolated node	BFS/DFS from it visits just itself, correctly representing a trivial connected component
Grid boundary conditions	Grid-as-graph traversal	Always explicitly bounds-check row/column indices before treating a cell as a valid neighbor

Common mistakes: marking a node visited at BFS **dequeue** time instead of **enqueue** time, causing the same node to be enqueued multiple times (still correct eventually, but wastes time/space and can cause subtle distance-tracking bugs); forgetting to iterate over all nodes to handle disconnected components when the problem requires counting components or full coverage.

SECTION 8 — Pros & Cons

Advantages: O(V + E) — optimal, since every vertex and edge must be examined at least once for full traversal-based questions; BFS gives shortest paths "for free" in unweighted graphs; DFS naturally supports recursive structural analysis (cycles, connectivity, articulation points). **Disadvantages:** BFS's queue can grow to O(V) in the worst case (wide graphs); DFS's recursion depth can reach O(V) for deep/skewed graphs, risking stack overflow. **Trade-offs:** BFS (shortest path in unweighted graphs, more memory for wide graphs) vs. DFS (structural analysis, more memory for deep graphs, natural recursion) — choose based on whether the question is about distance (BFS) or structure (DFS). **Limitations:** Neither directly handles weighted shortest-path (needs Dijkstra's/Bellman-Ford); DFS-based cycle detection differs meaningfully between directed (3-color marking) and undirected (simple visited + parent-tracking) graphs. **Inefficient when:** using an adjacency matrix representation for a sparse graph ($O(V^2)$ instead of O(V+E)) — always prefer adjacency lists when the graph is sparse.

SECTION 9 — Real World Applications

Domain	Application
Google	Web crawling (BFS-like exploration from seed URLs), PageRank-adjacent graph structure analysis
Meta/LinkedIn	"Degrees of separation," friend-of-friend suggestions via BFS from a user node
Amazon	Warehouse/delivery network connectivity and routing graph analysis
Uber/Grab	Road network shortest-path (unweighted hop-count) estimation before applying weighted routing
Networking	Network topology discovery and broadcast/flooding protocols (BFS-like propagation)
Operating Systems	Deadlock detection via cycle detection in resource-allocation graphs
Compilers	Dependency graph analysis (module/package dependency cycles) via DFS-based cycle detection
Social Network Analysis	Connected component detection (community detection), influence propagation modeling
Package Managers (npm, pip)	Dependency resolution and circular-dependency detection via graph traversal
Distributed Systems	Service mesh dependency graph analysis, detecting circular service call chains

---

SECTION 10 — Interview Strategy

How seniors answer: They immediately identify whether the graph may have cycles (requiring a visited set, unlike tree traversal), correctly choose BFS for unweighted shortest-path questions and DFS for structural questions, and mark nodes visited at the correct time (enqueue for BFS) to avoid subtle bugs.

How juniors answer: They sometimes apply tree-traversal code directly to a graph without adding a visited set, causing infinite loops on cyclic input, or they confuse BFS and DFS's appropriate use cases (e.g., using DFS for a shortest-path question, which doesn't guarantee correctness without additional bookkeeping).

Typical follow-ups: "What if the graph has weighted edges?" (Dijkstra's/Bellman-Ford). "How do you detect a cycle in a directed vs undirected graph — are the techniques different?" (Yes — 3-color DFS for directed, visited+parent-tracking or Union-Find for undirected). "Can you find all connected components?" "What if the graph is represented as an adjacency matrix instead of a list — how does complexity change?"

Optimization questions: "Can you do a bidirectional BFS to speed up shortest-path search?" (meet-in-the-middle technique, halving the effective search depth in some cases).

SECTION 11 — Pattern Variations

Variation	Description	Example
Grid-as-Graph BFS/DFS	Adjacent cells are edges	Number of Islands, Rotting Oranges, Flood Fill
Multi-Source BFS	Start BFS from multiple sources simultaneously	Rotting Oranges, Walls and Gates
Cycle Detection (Directed)	3-color DFS marking	Course Schedule (cycle check)
Cycle Detection (Undirected)	Visited + parent tracking, or Union-Find	Graph Valid Tree
Connected Components Counting	Iterate all nodes, BFS/DFS from each unvisited one	Number of Provinces, Number of Islands
Bipartite Graph Checking	BFS/DFS with 2-coloring	Is Graph Bipartite?
Clone Graph	DFS/BFS with a visited map storing node copies	Clone Graph
Bidirectional BFS	Simultaneous BFS from both source and target	Word Ladder (optimization)

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
Tree DFS/BFS	No visited set needed (trees are acyclic by definition)	Data is a tree, not a general graph
Union-Find	Efficient incremental connectivity/cycle detection for undirected graphs, especially with many edge-addition queries	Repeated “are these connected” queries as edges are added dynamically
Topological Sort	Specifically produces a valid dependency ordering for DAGs	Need an ordering respecting “must come before” constraints
Dijkstra’s/Bellman-Ford	Handles weighted edges for shortest path	Edge weights aren’t uniform

Comparison Table

Aspect	Graph BFS	Graph DFS	Union-Find
Best for	Shortest path (unweighted), level exploration	Connectivity, cycle detection, exhaustive paths	Incremental connectivity, cycle detection (undirected)
Time	O(V+E)	O(V+E)	$O(E \times \alpha(V)) \approx O(E)$
Natural recursion	No (queue-based)	Yes (or explicit stack)	No

---

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	Basic connectivity, flood fill	Flood Fill, Find if Path Exists in Graph
Medium	Grid-based BFS/DFS, component counting, cycle detection	Number of Islands, Number of Provinces, Course Schedule, Rotting Oranges
Hard	Bipartite checking, clone graph, multi-source BFS with constraints	Is Graph Bipartite?, Clone Graph, Word Ladder
Very Hard	Bidirectional BFS, complex multi-constraint graph traversal	Word Ladder II, Bus Routes, Sliding Puzzle

---

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	Number of Islands	Medium	Amazon, Meta, Microsoft, Google	Grid-as-graph DFS/BFS component counting	Foundational grid traversal
2	Flood Fill	Easy	Amazon, Meta	Grid-as-graph DFS/BFS	Basic grid traversal
3	Rotting Oranges	Medium	Amazon, Meta, Google	Multi-source BFS	Multi-source BFS mastery
4	Number of Provinces	Medium	Amazon, Meta	Connected components counting	Component counting mechanics
5	Course Schedule	Medium	Amazon, Meta, Google, Microsoft	Directed graph cycle detection	Cycle detection (directed)
6	Course Schedule II	Medium	Amazon, Meta, Google	Cycle detection + topological ordering	Cross-pattern (cycle detection + topo sort)
7	Clone Graph	Medium	Amazon, Meta, Google	DFS/BFS with visited map for node copies	Traversal with state mapping
8	Is Graph Bipartite?	Medium	Amazon, Google	BFS/DFS with 2-coloring	Bipartite checking mastery
9	Graph Valid Tree	Medium	Google, Amazon	Undirected cycle detection + connectivity	Cross-pattern (cycle + connectivity)

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
10	Walls and Gates	Medium	Amazon, Google	Multi-source BFS	Multi-source BFS reinforcement
11	Word Ladder	Hard	Amazon, Meta, Google, Microsoft	BFS shortest transforma-tion path	Advanced BFS application
12	Word Ladder II	Hard	Amazon, Google	BFS + path reconstruction	Advanced BFS + backtracking combination
13	Pacific Atlantic Water Flow	Medium	Amazon, Google	Multi-source DFS/BFS from boundary cells	Reverse multi-source traversal
14	Surrounded Regions	Medium	Amazon, Meta, Google	Boundary-based DFS/BFS marking	Boundary-exclusion traversal
15	01 Matrix	Medium	Amazon, Google	Multi-source BFS for distance computation	Multi-source distance BFS
16	Find if Path Exists in Graph	Easy	Amazon	Basic connectivity check	Foundational connectivity
17	Minimum Height Trees	Medium	Google, Amazon	BFS-based topological peeling (leaf removal)	Advanced BFS application
18	Bus Routes	Hard	Google, Amazon	BFS over a transformed graph (routes as nodes)	Advanced graph transformation
19	Sliding Puzzle	Hard	Google	BFS over a state-space graph	State-space BFS mastery
20	Open the Lock	Medium	Amazon, Google	BFS over a state-space graph with deadends	State-space BFS with constraints
21	Shortest Path in Binary Matrix	Medium	Amazon, Google	Grid BFS shortest path	Grid BFS shortest path
22	As Far from Land as Possible	Medium	Amazon, Google	Multi-source BFS for max-distance computation	Multi-source BFS variant

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
23	Reorder Routes to Make All Paths Lead to the City Zero	Medium	Amazon, Google	Directed graph DFS/BFS with edge-direction tracking	Directed traversal with edge tracking
24	Evaluate Division	Medium	Amazon, Google, Meta	Weighted graph DFS/BFS for ratio computation	Weighted graph traversal application
25	All Paths From Source to Target	Medium	Amazon, Google	DFS with path/backtracking for DAG enumeration	Cross-pattern (DFS + Backtracking)
26	Keys and Rooms	Medium	Amazon, Google	Basic DFS/BFS connectivity with dynamic unlocking	Connectivity with state dependencies
27	Redundant Connection	Medium	Amazon, Google	Cycle detection in undirected graph (Union-Find alternative)	Cross-pattern (DFS + Union-Find comparison)
28	Accounts Merge	Medium	Amazon, Meta, Google	Connected components via DFS/BFS or Union-Find	Cross-pattern (Graph + Union-Find)
29	Cheapest Flights Within K Stops (contrast, weighted)	Medium	Amazon, Google	Contrast: needs Bellman-Ford/Dijkstra's variant, not plain BFS	Pattern-boundary awareness
30	Network Delay Time (contrast, weighted)	Medium	Amazon, Google, Meta	Contrast: needs Dijkstra's, not plain BFS	Pattern-boundary awareness

---

SECTION 15 — Common Mistakes

1. Marking a node visited at BFS **dequeue** time instead of **enqueue** time, causing duplicate enqueues and potential incorrect distance tracking. *Fix:* always mark visited immediately upon enqueueing.
2. Forgetting to handle disconnected graphs — only running BFS/DFS from a single starting node when the problem requires covering all components. *Fix:* iterate over all nodes, starting a new traversal from each unvisited one.

3. Applying tree-traversal code directly to a graph without adding a visited set, causing infinite loops on cyclic input. *Fix:* always add and maintain a visited set for any graph (not tree) traversal.
4. Confusing directed and undirected cycle-detection techniques — using undirected logic (simple visited-set + parent-skip) on a directed graph, missing valid cycles, or vice versa. *Fix:* always clarify edge directionality and use the matching cycle-detection technique (3-color DFS for directed, visited+parent or Union-Find for undirected).
5. Using BFS for a weighted-edge shortest-path problem without recognizing that plain BFS assumes unit weights. *Fix:* recognize weighted edges as the signal to pivot to Dijkstra's/Bellman-Ford instead.

Why people fail: the addition of a visited set feels like a minor addition over tree traversal, but candidates who don't fully internalize *why* it's needed (cycles causing infinite loops) sometimes add it inconsistently (e.g., only checking it in one branch of the code), leading to subtle correctness bugs that only manifest on cyclic test cases, which interviewers often specifically include.

SECTION 16 — Optimization Techniques

- **Time:** Use adjacency lists (O(V+E)) instead of adjacency matrices ($O(V^2)$) for sparse graphs; use bidirectional BFS (searching simultaneously from source and target) to reduce effective search depth for shortest-path problems with a known target.
- **Space:** For very large graphs, consider iterative DFS with an explicit stack instead of recursion to avoid stack-overflow risk on deep/skewed graphs.
- **Readability:** Clearly separate "mark visited" from "process node" logic; use descriptive variable names (`visited`, `queue`, `adjacencyList`) rather than generic single-letter names.
- **Interview performance:** Explicitly state the O(V+E) complexity and justify the visited set's necessity (cycle prevention) — this small habit signals solid graph fundamentals.

SECTION 17 — Multiple Language Templates

Java

```java
public void bfs(Map<Integer, List<Integer>> graph, int source) {
    Set<Integer> visited = new HashSet<>();
    Queue<Integer> queue = new LinkedList<>();
    queue.offer(source);
    visited.add(source);
    while (!queue.isEmpty()) {
        int node = queue.poll();
        for (int neighbor : graph.getOrDefault(node, new ArrayList<>())) {
            if (!visited.contains(neighbor)) {
                visited.add(neighbor);
                queue.offer(neighbor);
            }
        }
    }
}
```

JavaScript

```javascript
function bfs(graph, source) {
    const visited = new Set([source]);
```

```
    const queue = [source];
    while (queue.length) {
        const node = queue.shift();
        for (const neighbor of graph[node] || []) {
            if (!visited.has(neighbor)) {
                visited.add(neighbor);
                queue.push(neighbor);
            }
        }
    }
}
```

PHP

```
function bfs(array $graph, $source): void {
    $visited = [$source => true];
    $queue = [$source];
    while (!empty($queue)) {
        $node = array_shift($queue);
        foreach ($graph[$node] ?? [] as $neighbor) {
            if (!isset($visited[$neighbor])) {
                $visited[$neighbor] = true;
                $queue[] = $neighbor;
            }
        }
    }
}
```

Python

```
from collections import deque
def bfs(graph, source):
    visited = {source}
    queue = deque([source])
    while queue:
        node = queue.popleft()
        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

Go

```
func bfs(graph map[int][]int, source int) {
    visited := map[int]bool{source: true}
    queue := []int{source}
    for len(queue) > 0 {
        node := queue[0]
        queue = queue[1:]
        for _, neighbor := range graph[node] {
            if !visited[neighbor] {
                visited[neighbor] = true
                queue = append(queue, neighbor)
```

```
            }
        }
    }
}
```

C++

```cpp
void bfs(unordered_map<int, vector<int>>& graph, int source) {
    unordered_set<int> visited{source};
    queue<int> q;
    q.push(source);
    while (!q.empty()) {
        int node = q.front(); q.pop();
        for (int neighbor : graph[node]) {
            if (!visited.count(neighbor)) {
                visited.insert(neighbor);
                q.push(neighbor);
            }
        }
    }
}
```

SECTION 18 — Dry Runs

Small Input

Graph: `1-2, 1-4, 2-3, 4-5, 3-5` (matches §1.5)

```
BFS from 1:
visited={1}, queue=[1]
dequeue 1: neighbors 2,4 both unvisited → visited={1,2,4}, queue=[2,4]
dequeue 2: neighbor 3 unvisited → visited={1,2,4,3}, queue=[4,3]
dequeue 4: neighbor 5 unvisited (1 already visited) → visited={1,2,4,3,5}, queue=[3,5]
dequeue 3: neighbor 5 already visited → queue=[5]
dequeue 5: neighbors 3,4 already visited → queue=[]
BFS visit order: 1,2,4,3,5
```

Large Input (Conceptual)

For a graph with 10^6 vertices and 5×10^6 edges (sparse), BFS/DFS with an adjacency list visits each vertex once and examines each edge once — O(6×10^6) total operations, versus O((10^6)²) if an adjacency matrix were used instead — an enormous practical difference for sparse graphs.

Corner Case

Disconnected graph: nodes `{1,2}` connected, node `{3}` isolated. BFS from 1 visits only `{1,2}`; a separate traversal must start from 3 to visit it, correctly identifying 2 connected components total.

SECTION 19 — Advanced Concepts

- **Bidirectional BFS:** for shortest-path problems with a known source AND target, simultaneously growing BFS frontiers from both ends and stopping when they meet can reduce the

effective search space from O(b^d) to O(b^(d/2)) (b = branching factor, d = distance) — a significant practical speedup for problems like Word Ladder.
- **Multi-source BFS:** initializing the BFS queue with multiple starting nodes simultaneously (e.g., all initially-rotten oranges, or all land cells in "01 Matrix") correctly computes the shortest distance from the *nearest* source to every other node, in the same O(V+E) complexity as single-source BFS.
- **3-color DFS for directed cycle detection:** the "gray" (currently in recursion stack) vs "black" (fully processed) distinction is essential for directed graphs — a "black" node reachable via a different path is NOT a cycle (it's just a shared descendant, common in DAGs), but a "gray" node reached again IS a cycle (a genuine back-edge to an ancestor in the current recursion).
- **Graph traversal on implicit/generated graphs:** many "hard" problems (Word Ladder, Sliding Puzzle, Open the Lock) don't give you an explicit graph — you must recognize that "states" are nodes and "valid transitions" are edges, then apply standard BFS/DFS to this implicit, often much larger, state-space graph.

SECTION 20 — Senior Engineer Insights

Staff engineers recognize graph BFS/DFS as **the** foundational toolkit for any connectivity, reachability, or shortest-unweighted-path question — and they're fluent at recognizing *implicit* graphs (state spaces, transformation sequences, dependency chains) that don't superficially look like graphs but are best solved by modeling states as nodes and valid transitions as edges. They also know precisely when plain BFS/DFS is insufficient (weighted edges, ordering constraints) and pivot immediately to the appropriate specialized algorithm (Dijkstra's, Topological Sort, Union-Find) rather than trying to force-fit BFS/DFS onto a fundamentally different problem shape. Interviewers evaluate this recognition ability — modeling an unusual problem as an implicit graph — as a hallmark of strong algorithmic maturity.

SECTION 21 — Cheat Sheet

```
PATTERN: Graph Traversal (BFS & DFS)
RECOGNIZE: "connected components," "islands," "shortest path" (unweighted), "path exists," grid movement p
TEMPLATE (BFS):
    visited = {source}; queue = [source]
    while queue: node = dequeue()
        for neighbor in adj[node]:
            if neighbor not in visited: visited.add(neighbor); enqueue(neighbor)
TEMPLATE (DFS):
    function dfs(node, visited):
        if node in visited: return
        visited.add(node); process(node)
        for neighbor in adj[node]: dfs(neighbor, visited)
COMPLEXITY: O(V + E) time and space (adjacency list representation)
KEY PROOF: visited set guarantees each node/edge processed once; BFS's FIFO order guarantees shortest unwe
WATCH FOR: mark-visited timing (enqueue not dequeue for BFS), disconnected components, directed vs undirec
DOESN'T APPLY WHEN: weighted edges (use Dijkstra's/Bellman-Ford), need dependency ordering (use Topologica
```

SECTION 22 — Revision Notes (5-Minute Review)

- Graph traversal = Tree traversal + mandatory visited set (cycles possible).
- BFS → shortest path in unweighted graphs (level-synchronized, FIFO guarantees this).

- DFS → connectivity, cycle detection, exhaustive path enumeration.
- Mark visited at BFS enqueue time, not dequeue time.
- Iterate all nodes to handle disconnected components.
- Directed cycle detection needs 3-color DFS (white/gray/black); undirected needs visited+parent or Union-Find.
- Many "hard" problems are implicit graphs (state-space BFS) — recognize states-as-nodes, transitions-as-edges.

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Basic grid/graph traversal	Flood Fill (733), Find if Path Exists in Graph (1971)
Intermediate	Component counting, multi-source BFS	Number of Islands (200), Number of Provinces (547), Rotting Oranges (994)
Advanced	Cycle detection, bipartite checking	Course Schedule (207), Is Graph Bipartite? (785), Graph Valid Tree (261)
Expert	Implicit state-space graphs, bidirectional BFS	Word Ladder (127), Sliding Puzzle (773), Bus Routes (815)

SECTION 24 — Memory Tricks

- **Mnemonic:** "**M**ark before you **Move**" — always mark a node visited before/as you enqueue or recurse into it.
- **Visualization: Chalk-marking junctions in a maze with cycles** — without marks, you loop forever; with them, every room is explored exactly once.
- **Recognition shortcut:** Grid movement, "connected," "islands," "path exists" → Graph BFS/DFS; unweighted + "shortest" → BFS specifically; "cycle/structure" → DFS specifically.

SECTION 25 — Final Summary

Graph BFS and DFS generalize tree traversal to structures with cycles by adding a mandatory visited set, guaranteeing O(V+E) traversal that visits every reachable node and edge exactly once. The single most important thing to remember forever: **BFS's FIFO queue guarantees the first time you reach a node is via the shortest unweighted path — but only if you mark nodes visited at enqueue time, not dequeue time — and DFS's depth-first commitment naturally suits connectivity and cycle-detection questions, with directed and undirected cycle detection requiring genuinely different techniques (3-color marking vs. visited+parent-tracking).** Many of the hardest graph problems aren't explicitly graphs at all — recognizing an implicit state-space (valid transformations as edges, states as nodes) and applying standard BFS/DFS to it is a core advanced skill.